

--- Unit-I: Front End Tools ---

1. Explain the structure of a basic HTML5 document

A basic HTML5 document follows a specific hierarchy to ensure the browser interprets the page correctly:

- **<!DOCTYPE html>:** A declaration that tells the browser the document is HTML5.
- **<html>:** The root element that wraps all content.
- **<head>:** Contains metadata, such as the page title, character set (UTF-8), and links to CSS files. This information isn't visible on the page.
- **<body>:** Contains the visible content of the webpage, including text, images, and links.

2. What are HTML5 semantic elements?

Semantic elements are tags that clearly describe their meaning to both the browser and the developer. Before HTML5, developers used `<div>` tags for everything. Now, elements like `<header>`, `<nav>`, `<main>`, `<article>`, and `<footer>` define the layout structure. This improves **SEO** (Search Engine Optimization) and **accessibility** for screen readers.

3. Describe the three types of CSS styling

CSS can be applied to HTML in three ways:

- **Inline:** Added directly to an element using the style attribute (e.g., `<p style="color:red;">`). Use this sparingly.
- **Internal:** Defined within `<style>` tags inside the `<head>` of an HTML document. Good for single-page styles.
- **External:** Written in a separate .css file and linked in the `<head>`. This is the professional standard as it allows you to change the look of an entire website from one file.

4. What is the Bootstrap Grid System?

The **Bootstrap Grid System** is a mobile-first, flexbox-based layout tool. It divides the page into a **12-column** grid. You can specify how many columns an element should take up depending on the screen size (extra small, small, medium, large, etc.). This makes creating responsive websites much faster.

5. What is the purpose of the 'alt' attribute in images?

The alt (alternative text) attribute serves two vital roles:

1. **Accessibility:** Screen readers read the text aloud to visually impaired users.
2. **Fallback:** If the image fails to load due to a slow connection or broken link, the text appears in its place so the user still understands the context.

6. Difference between <div> and ?

- **<div> (Division):** A **block-level** element. It always starts on a new line and takes up the full width available. It is used to group large sections of content.
- **:** An **inline** element. It does not start on a new line and only takes up as much width as necessary. It is used to style specific parts of a text (like one word inside a paragraph).

7. How do you create a table in HTML?

Tables are built row by row using these tags:

- **<table>:** The container for the table.
- **<tr> (Table Row):** Defines a horizontal row.
- **<th> (Table Header):** Defines a header cell (usually bold and centered).
- **<td> (Table Data):** Defines a standard data cell.

8. What is the use of the <form> element?

The `<form>` element acts as a container for different types of input, such as text fields, checkboxes, and radio buttons. It collects this user data and sends it to a server for processing (like logging in or signing up) when the user clicks a "Submit" button.

9. What is XML?

XML (Extensible Markup Language) is a markup language designed to store and transport data. Unlike HTML, which focuses on how data *looks*, XML focuses on what the data *is*. It allows you to create your own tags (e.g., <book>, <price>), making it a flexible way to share data between different types of software.

10. Define JSON and its syntax

JSON (JavaScript Object Notation) is the most popular lightweight format for exchanging data between a server and a web application. It is easy for humans to read and write and easy for machines to parse.

- **Syntax:** Data is stored in **key/value pairs** separated by commas.
- **Example:** {"city": "Pune", "population": 3100000}. Arrays are stored in square brackets [], and objects in curly braces {}.

--- Unit-II: Client-Side Technologies ---

1. What are the primary JavaScript data types?

JavaScript data types are divided into two categories: **Primitives** and **Objects**.

- **Primitives:** These are immutable and include String (text), Number (integers and decimals), Boolean (true/false), Null (intentional absence of value), Undefined (variable declared but not assigned), and Symbol (unique identifiers).
- **Objects:** A complex data type used to store collections of data and more complex entities (including Arrays and Functions).

2. Explain 'Scopes' in JavaScript

Scope determines the accessibility of variables:

- **Global Scope:** Variables declared outside any function or block; accessible from anywhere in the script.
- **Function Scope:** Variables declared inside a function; accessible only within that function.
- **Block Scope:** Introduced with let and const. Variables declared inside curly braces {} (like an if statement or a for loop) cannot be accessed from outside those braces.

3. How is form validation handled in HTML5?

HTML5 allows for "client-side" validation without writing JavaScript, using specific attributes:

- **required:** Prevents form submission if the field is empty.
- **type="email" / "url":** Browsers automatically check for valid formatting (e.g., presence of "@" in email).
- **pattern:** Allows you to use **Regular Expressions (Regex)** to enforce specific formats, like a custom ZIP code or password strength.
- **min/max/step:** Essential for numeric or date inputs to ensure data falls within a specific range.

4. What is the DOM?

The **Document Object Model (DOM)** is a programming interface for web documents. When a browser loads a webpage, it creates a **tree-like structure** of the HTML. JavaScript uses the DOM to dynamically change the content, style, and structure of the page.

5. Name the DOM levels

The DOM has evolved through several versions or "Levels" standardized by the W3C:

- **Level 0:** The original, non-standardized way to access elements (e.g., document.forms).
- **Level 1:** The first standard that allowed navigating and changing document structure.
- **Level 2:** Added support for **XML namespaces**, CSS style manipulation, and the standard **Event Model** (e.g., addEventListener).
- **Level 3:** Introduced advanced features like XPath and better keyboard event handling.
- **Living Standard:** Today, the DOM is maintained as a "Living Standard" that is updated continuously rather than in discrete levels.

6. How does jQuery select elements?

jQuery uses a "Sizzle" engine to select elements using **CSS Selectors**. It starts with the \$() factory function.

- **Class:** \$('my-class') selects all elements with that class.
- **ID:** \$('#my-id') selects the single element with that ID.

- **Tag:** \$('p') selects all paragraph elements.
This makes selecting complex nested elements much easier than using raw JavaScript.
-

7. What is the Flexbox layout?

Flexbox (Flexible Box Layout) is a 1D (one-dimensional) layout model. It allows items within a container to automatically adjust their width, height, and order to best fill the available space. It is perfect for aligning items horizontally or vertically and handling different screen sizes.

8. Difference between innerHTML and textContent?

- **innerHTML:** Returns or sets the HTML markup contained within an element. If you set innerHTML = "Hi", it will render **Hi** in bold. **Warning:** This can be a security risk (XSS attacks) if used with user input.
- **textContent:** Returns or sets the text content of the node and its descendants. If you set textContent = "Hi", the browser will literally display the tags Hi on the screen.

9. How do you handle events in jQuery?

jQuery simplifies event handling across different browsers.

- **The .on() method:** The modern, preferred way to attach event handlers (e.g., \$('button').on('click', function() { ... })).
- **Shorthand methods:** Methods like .click(), .hover(), and .submit() are shortcuts for .on('event', ...).
- **Delegation:** jQuery makes it easy to handle events for elements that haven't been created yet (dynamic content).

10. What is the purpose of Postman?

Postman is a tool used by developers to build, test, and document **APIs**. Instead of writing frontend code just to see if your backend works, you can use Postman to send HTTP requests (GET, POST, etc.) to a URL and see the exact JSON or HTML response the server sends back. It is essential for debugging server-side logic.

--- Unit-III: Server-Side Technologies ---

1. How does a PHP script execute?

PHP is a **server-side scripting language**. When a user requests a PHP page, the server (like Apache or Nginx) passes the file to the PHP interpreter. The interpreter executes the code, potentially fetching data from a database, and generates a plain HTML response. This HTML is then sent back to the user's browser. The user never sees the actual PHP code, only the result it produces.

2. Difference between Sessions and Cookies in PHP

- **Cookies:** These are small files stored on the **client's (user's) computer**. They are used to track preferences or "remember" users. Since they are stored locally, users can modify or delete them.
- **Sessions:** These store data on the **server**. A unique "Session ID" is given to the user (often via a cookie), which acts like a key to unlock their specific data on the server. Because the actual data stays on the server, sessions are much more secure for sensitive info like login status.

3. Explain \$_GET vs \$_POST

These are "superglobals" used to collect form data:

- **\$_GET:** Data is appended to the URL (e.g., search.php?query=book). It is visible to everyone, can be bookmarked, and has character limits.
 - **\$_POST:** Data is sent inside the body of the HTTP request. It is invisible in the URL, has no size limit, and is the standard for sensitive data like passwords.
-

4. How do you connect PHP to MySQL?

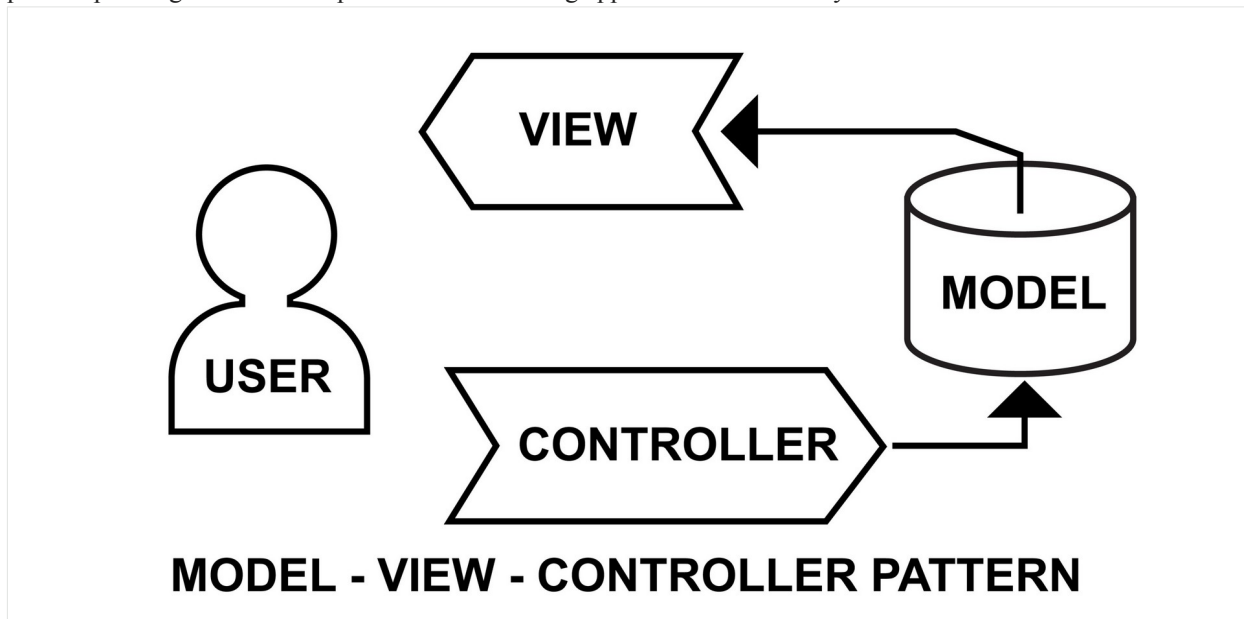
There are two modern ways:

- **MySQLi (MySQL Improved):** Designed specifically for MySQL databases. It supports both object-oriented and procedural programming.

- **PDO (PHP Data Objects):** A universal layer. If you use PDO, you can switch from MySQL to PostgreSQL or SQLite by changing only a few lines of code.

5. What is Laravel?

Laravel is a powerful **PHP framework** that follows the **MVC (Model-View-Controller)** design pattern. It provides built-in tools for common tasks like authentication, routing, sessions, and caching. Its goal is to make the development process pleasing for the developer without sacrificing application functionality.



Shutterstock

6. How is Exception Handling done in PHP?

Exception handling is used to change the normal flow of a code if a specified error condition occurs.

- **Try:** The block of code where an error might occur.
- **Throw:** How you trigger an exception.
- **Catch:** The block that "catches" the exception and handles the error gracefully so the whole site doesn't crash.

7. Name functions for file handling in PHP

PHP treats files as resources. The standard workflow is:

1. **fopen():** Opens a file or URL.
2. **fread():** Reads a specific number of bytes from the file.
3. **fwrite():** Writes data into the file.
4. **fclose():** Closes the file to save memory and prevent errors.

8. What is Web 3.0?

Web 3.0 is the third generation of internet services. While Web 2.0 was about social media and user-generated content (centralized in companies like Google/Meta), **Web 3.0** focuses on **decentralization**. It uses **blockchain technology**, smart contracts, and decentralized data storage to give users more control over their own data and identity.

9. How do you validate email in PHP?

Rather than writing complex regular expressions, PHP has a built-in function called `filter_var()`. By using the constant `FILTER_VALIDATE_EMAIL`, PHP automatically checks if the string follows the correct syntax (e.g., has an "@" and a valid domain extension).

10. What is an associative array?

In a standard array, items are accessed by a number (0, 1, 2). In an **associative array**, you assign **custom keys** to values. This makes the data much more descriptive.

- *Example:* `$user = array("name" => "John", "age" => 25);`
- Instead of `$user[0]`, you use `$user["name"]`.

--- Unit-IV: Spring Boot ---

1. What is Spring Boot?

Spring Boot is an extension of the Spring framework that simplifies the process of developing Java applications. It follows an "**opinionated**" approach, meaning it comes with pre-configured settings so you can "just run" your application without spending days on manual configuration. Its main goal is to reduce boilerplate code and get a production-ready app up and running quickly.

2. What is Inversion of Control (IoC)?

IoC is a design principle where the control of objects or portions of a program is transferred to a container or framework. In a traditional program, you (the developer) create an object using the `new` keyword. In Spring, the **Spring IoC Container** creates the object, wires it to other objects, and manages its entire life cycle. This makes your code more modular and easier to test.

3. What are Maven and Gradle?

These are **build automation tools**. They use a configuration file (`pom.xml` for Maven, `build.gradle` for Gradle) to:

- **Manage Dependencies:** Automatically download libraries (like Spring or Hibernate) from the internet.
 - **Compile Code:** Turn your Java files into bytecode.
 - **Package Apps:** Bundle your code into a JAR or WAR file for deployment.
-

4. What does Spring Security provide?

Spring Security is a powerful framework focusing on providing both **authentication** (who are you?) and **authorization** (what are you allowed to do?). It also provides built-in protection against common attacks like **CSRF** (Cross-Site Request Forgery) and **Session Fixation**, making it the standard for securing Java-based web applications.

5. Explain JPA in Spring Boot

The **Java Persistence API (JPA)** is a specification that allows you to interact with databases using Java objects instead of writing raw SQL. In Spring Boot, you typically use **Spring Data JPA**. It maps your Java classes (Entities) to database tables, and your class variables to table columns. This process is called **Object-Relational Mapping (ORM)**.

6. What is an Embedded Server?

In traditional Java web development, you had to install a server (like Tomcat) on your computer and then "deploy" your app to it. Spring Boot includes the server **inside** the application's `.jar` file. When you run your app, the server starts automatically. This makes deployment much easier because you only need a Java Runtime Environment (JRE) to run your app anywhere.

7. SQL vs NoSQL

- **SQL (Relational):** Data is stored in structured tables with fixed rows and columns. It uses **ACID** properties to ensure reliability. Examples: MySQL, PostgreSQL.
- **NoSQL (Non-Relational):** Data can be stored as documents, key-value pairs, or graphs. It is highly scalable and flexible for unstructured data. Example: MongoDB.

8. What is a Spring Boot Starter?

Starters are a set of convenient dependency descriptors. Instead of manually searching for and adding 10 different library versions to your project, you add one "Starter." For example, `spring-boot-starter-web` automatically includes everything you need to build a web app, including Spring MVC, Jackson (for JSON), and an embedded Tomcat server.

9. How do you test Spring Boot apps?

Spring Boot provides the `spring-boot-starter-test`, which includes:

- **JUnit:** The standard framework for writing unit tests.
- **Mockito:** A tool used to "mock" (simulate) dependencies so you can test a single component in isolation.
- **@SpringBootTest:** An annotation that tells Spring to start the application context so you can perform integration testing.

10. What is @SpringBootApplication?

This is the most important annotation in a Spring Boot project. It is actually a **convenience annotation** that combines three others:

1. **@Configuration:** Marks the class as a source of bean definitions.
2. **@EnableAutoConfiguration:** Tells Spring Boot to start adding beans based on the libraries present on your classpath.
3. **@ComponentScan:** Tells Spring to look for other components (like Controllers or Services) in the current package and its sub-packages.

--- Unit-V: React ---

1. What is JSX?

JSX (JavaScript XML) is a syntax extension for JavaScript. It looks like HTML but is actually converted into regular JavaScript objects behind the scenes. It makes code easier to write and read by allowing you to structure your UI directly within your logic. For example, `<h1>Hello</h1>` in JSX is transformed into a `React.createElement()` call.

2. Name the React Lifecycle Phases

Components go through a "lifecycle" from birth to death:

- **Mounting:** Putting the component into the DOM (the browser's view).
- **Updating:** When a component's props or state change, it re-renders to show the new data.
- **Unmounting:** Removing the component from the DOM (e.g., navigating to a different page).

3. Props vs State

Both hold information that influences the UI, but their "ownership" differs:

- **Props (Properties):** These are like function arguments. They are passed from a parent to a child and are **read-only**.
- **State:** This is the component's private data. It is **mutable** (can be changed) and lives inside the component. When state changes, the component re-renders.

4. How is data passed from Child to Parent?

In React, data flows down (Parent to Child). To send data "up," the Parent passes a **callback function** as a prop to the Child. The Child then calls that function and passes the data as an argument, which the Parent receives and processes.

5. What are React Hooks?

Before version 16.8, you needed Class components to use state. **Hooks** are functions that let you "hook into" React state and lifecycle features from functional components.

- **useState:** Manages local data.
- **useEffect:** Handles side effects like fetching data or setting timers.

6. What is Redux?

Redux is a pattern and library for managing "global" state. Instead of passing data through ten layers of components (prop drilling), you store all important data in a single **Store**. Any component in the app can then "subscribe" to the store to get exactly what it needs.

7. What is the use of 'Keys' in React?

When you render a list of items, React needs a way to track which item is which. **Keys** (usually unique IDs) help React identify which specific items have changed, been added, or been removed. This significantly improves performance because React only updates the specific element that changed instead of re-rendering the entire list.

8. What is React Router?

React is used for **Single Page Applications (SPAs)**, where the page never actually refreshes. **React Router** is the standard library that manages "routes." It syncs the URL in the browser address bar with the components displayed on the screen, allowing for seamless navigation (e.g., `/profile` vs `/settings`).

9. Functional vs Class Components

- **Class Components:** Older style using ES6 classes. They require this keyword and lifecycle methods like `componentDidMount`.
- **Functional Components:** Modern, simpler JavaScript functions. Since the introduction of Hooks, they can do everything Class components can do with less code and better performance.

10. What is Component Composition?

Instead of building one giant, messy component, you build small, specialized ones (like a Button, Input, or Card) and "compose" them together. This is often achieved using the **children prop**, allowing you to nest components inside each other just like HTML tags. This makes your code highly reusable and easier to maintain.

--- Unit-VI: Node JS ---

1. What is Node.js?

Node.js is an open-source, cross-platform JavaScript runtime environment. Historically, JavaScript could only run inside web browsers to make pages interactive. Node.js changed this by extracting Google Chrome's V8 engine—which compiles JavaScript directly into machine code—and allowing it to run on servers. Its architecture is event-driven and non-blocking, meaning it can handle many concurrent connections without waiting for one task to finish before starting the next.

2. What is NPM?

Node Package Manager (NPM) is the world's largest software registry. It serves two main purposes:

- A Command Line Interface (CLI): Used to install, update, and uninstall packages.
- An Online Repository: A place where developers share open-source code "packages" (like libraries for encryption, math, or UI components) so others don't have to reinvent the wheel.

3. Explain the Event Loop

The Event Loop is the secret behind Node.js's ability to be "multi-tasking" despite being single-threaded. Instead of creating a new thread for every request (which consumes high memory), Node.js delegates heavy tasks (like database queries or file reading) to the system kernel or a thread pool. Once that task is finished, the result is placed in a queue. The Event Loop constantly checks this queue and executes the associated callback function when the main thread is free.

4. What is the 'fs' module?

The File System (fs) module is a built-in tool that allows Node.js to interact with the computer's operating system to manage files. It supports both Synchronous (blocking) and Asynchronous (non-blocking) methods.

- `fs.readFile()`: Reads the content of a file.
- `fs.writeFile()`: Creates or replaces a file with new data.
- `fs.unlink()`: Deletes a file.

5. What is Express.js?

While you can build a server using just Node.js, it is tedious. Express.js is a "minimalist" framework built on top of Node.js. It simplifies the process of:

- Routing: Directing different URLs (e.g., /home, /login) to specific functions.
- Handling Requests: Easily processing GET, POST, PUT, and DELETE actions.
- Standardization: Providing a structured way to build scalable web applications and APIs.

6. How do you create a server in Node?

In core Node.js, you use the built-in http module. The `http.createServer()` method takes a callback function with two arguments: `req` (the incoming request) and `res` (the outgoing response).

JavaScript

```
const http = require('http');

http.createServer((req, res) => {
  res.write('Hello World');
  res.end();
}).listen(3000);
```

7. What are Node Modules?

Modules are like Lego blocks for your application—they are isolated blocks of code.

- Core Modules: Built-in tools like `fs`, `path`, and `http`.
- Local Modules: Files you create in your project.
- Third-Party Modules: Packages installed via NPM (e.g., `Express`).

You use `module.exports` to share code and `require()` to bring it into another file.

8. What is Middleware in Express?

Middleware is a function that sits between the request coming in and the final response going out. It has access to the Request object, the Response object, and the next function.

- Common uses: Checking if a user is logged in, logging the time of a request, or parsing JSON data from a form.
- Flow: Request → Middleware 1 → Middleware 2 → Route Handler → Response.

9. What is package-lock.json?

When you install a dependency, `package.json` might list a version like `^1.0.0`. The `^` means "install this version or any compatible newer version."

The `package-lock.json` file records the exact version (e.g., `1.0.4`) that was installed at that moment. This ensures that if another developer downloads your project, they get the exact same dependency tree, preventing "it works on my machine" bugs.

10. How do you connect Node to a database?

Node.js doesn't have a built-in database; it uses drivers or ORMs/ODMs (Object Relational/Data Mappers) to communicate with them:

- Mongoose: Used for MongoDB (NoSQL). It allows you to define "schemas" (structures) for your data.
- Sequelize: Used for SQL databases (MySQL, PostgreSQL). It allows you to interact with your database using JavaScript objects instead of writing raw SQL queries.